Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 2
по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ алгоритмов умножения матриц

Студент Дудырев Д.С.

Группа ИУ7-52Б

Преподаватель Волкова Л.Л.

Преподаватель Строганов Д.М.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**
1.1 Постановка задачи 5
1.2 Стандартный алгоритм умножения матриц 5
1.3 Алгоритм Винограда 5

2 Конструкторская часть **7**
2.1 Требования к программному обеспечению 7
2.2 Разработка алгоритмов 7
2.2.1 Стандартный алгоритм умножения матриц 8
2.2.2 Алгоритм Винограда 10
2.2.3 Оптимизированный алгоритм Винограда 13

3 Технологическая часть **21**
3.1 Средства реализации 21
3.2 Реализация алгоритмов 21
3.3 Тестирование 27

4 Исследовательская часть **29**
4.1 Модель вычислений 29
4.2 Трудоёмкость алгоритмов 30
4.2.1 Стандартный алгоритм умножения матриц 30
4.2.2 Алгоритм Винограда 30
4.2.3 Оптимизированный алгоритм Винограда 31
4.2.4 Сводная оценка для квадратных матриц 32
4.3 Характеристики устройства 33

4.4 Инструменты замера процессорного времени 33
4.5 Результаты замеров 34
4.6 Сравнительный анализ 36
4.7 Вывод 37

ЗАКЛЮЧЕНИЕ **39**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **41**

ПРИЛОЖЕНИЕ А **42**

ВВЕДЕНИЕ

Цель работы — выполнить сравнительный анализ алгоритмов умножения матриц по трудоёмкости.

Задачи работы:

1) описать решаемую согласно варианту задачу;
2) разработать алгоритмы решения задачи согласно варианту: стандартный алгоритм умножения матриц, алгоритм Винограда и две оптимизированные версии алгоритма Винограда по варианту;
3) описать разработанные алгоритмы схемами алгоритмов;
4) описать требования к программному обеспечению;
5) описать средства реализации алгоритмов;
6) реализовать разработанные алгоритмы;
7) выполнить тестирование реализации алгоритмов;
8) описать модель вычислений для оценки трудоёмкости алгоритмов и выполнить теоретический расчёт трудоёмкости;
9) описать инструменты замера процессорного времени выполнения реализаций алгоритмов;
10) выполнить массированные замеры процессорного времени выполнения реализаций алгоритмов в зависимости от линейного размера квадратных матриц, отдельно для чётных и нечётных размеров;
11) выполнить сравнительный анализ алгоритмов по трудоёмкости и по данным выполненных замеров.

1 Аналитическая часть

1.1 Постановка задачи

Даны матрица A размера $n \times m$ и матрица B размера $m \times q$ с целочисленными элементами. Требуется вычислить их произведение — матрицу $C = A \cdot B$ размера $n \times q$, элементы которой определяются формулой [1, с. 172]

$$c_{i,j} = \sum_{k=0}^{m-1} a_{i,k} \cdot b_{k,j}, \quad i = \overline{0, n-1}, \quad j = \overline{0, q-1}. \tag{1.1}$$

Произведение определено только в том случае, когда число столбцов матрицы A равно числу строк матрицы B.

Для решения задачи рассматриваются следующие алгоритмы:

— стандартный алгоритм умножения матриц;

— алгоритм Винограда;

— оптимизированный согласно варианту алгоритм Винограда в двух реализациях: с заменой деления на 2 побитовым сдвигом и с заменой присваиваний вида $x = x + y$ составным оператором $x \mathrel{+}= y$.

1.2 Стандартный алгоритм умножения матриц

Стандартный алгоритм вычисляет каждый элемент $c_{i,j}$ непосредственно по формуле (1.1) как скалярное произведение i-й строки матрицы A на j-й столбец матрицы B [1, с. 172–173]. Для этого используются три вложенных цикла: по строкам A, по столбцам B и по индексу суммирования k. На вычисление каждого из nq элементов результата затрачивается m умножений и m сложений, поэтому для квадратных матриц размера $N \times N$ временная сложность алгоритма составляет $O(N^3)$ [1, с. 173].

1.3 Алгоритм Винограда

Алгоритм Винограда [2] основан на том, что каждый элемент результата является скалярным произведением двух векторов, и это произведение можно переписать так, чтобы часть умножений выполнялась заранее, один раз для каждой строки и каждого столбца. Для векторов $V = (v_0, v_1, v_2, v_3)$ и

$W = (w_0, w_1, w_2, w_3)$ справедливо

$$V \cdot W = (v_0+w_1)(v_1+w_0)+(v_2+w_3)(v_3+w_2)-v_0v_1-v_2v_3-w_0w_1-w_2w_3. \quad (1.2)$$

Слагаемые $v_0v_1 + v_2v_3$ зависят только от строки матрицы A, а $w_0w_1 + w_2w_3$ — только от столбца матрицы B, поэтому их можно вычислить заранее. Для каждой строки A и каждого столбца B вычисляются

$$mulH_i = \sum_{k=0}^{\lfloor m/2 \rfloor - 1} a_{i,2k} \cdot a_{i,2k+1}, \quad i = \overline{0, n-1}, \quad (1.3)$$

$$mulV_j = \sum_{k=0}^{\lfloor m/2 \rfloor - 1} b_{2k,j} \cdot b_{2k+1,j}, \quad j = \overline{0, q-1}, \quad (1.4)$$

после чего элементы результата находятся по формуле

$$c_{i,j} = -mulH_i - mulV_j + \sum_{k=0}^{\lfloor m/2 \rfloor - 1} (a_{i,2k} + b_{2k+1,j}) \cdot (a_{i,2k+1} + b_{2k,j}). \quad (1.5)$$

При нечётном m последний элемент строки и столбца в пары не попадает, поэтому к каждому элементу результата дополнительно прибавляется слагаемое $a_{i,m-1} \cdot b_{m-1,j}$.

В основном цикле на каждую пару элементов выполняется одно умножение вместо двух в стандартном алгоритме, то есть число умножений в основной части уменьшается примерно вдвое ценой увеличения числа сложений и дополнительной памяти под массивы $mulH$ и $mulV$ размера n и q. Асимптотическая сложность алгоритма остаётся равной $O(N^3)$.

2 Конструкторская часть

2.1 Требования к программному обеспечению

1) программа загружает матрицы A и B из текстовых файлов, пути к которым вводит пользователь; одна строка файла соответствует одной строке матрицы, элементы — целые числа, разделённые пробелами;
2) при загрузке программа проверяет, что все элементы являются целыми числами, а все строки матрицы содержат одинаковое число элементов; при ошибке выводится сообщение, а ранее загруженная матрица сохраняется;
3) перед умножением программа проверяет, что число столбцов матрицы A равно числу строк матрицы B;
4) программа циклически выводит меню со следующими пунктами:
   - выполнение умножения выбранным алгоритмом;
   - выполнение умножения всеми алгоритмами;
   - вывод загруженных матриц;
   - выбор алгоритма;
   - загрузка матрицы A из файла;
   - загрузка матрицы B из файла;
   - замеры процессорного времени;
   - завершение работы;
5) в режиме замеров программа для каждого линейного размера N из набора $10, 20, \ldots, 200$ (чётные) и $11, 21, \ldots, 201$ (нечётные) генерирует случайные квадратные матрицы, выполняет их умножение каждым алгоритмом, выводит среднее процессорное время на экран и сохраняет результаты в файлы формата CSV, по одному на алгоритм.

2.2 Разработка алгоритмов

Для каждого алгоритма определены:

— a — матрица A размера $n \times m$;

— b — матрица B размера $m \times q$;

— c — результирующая матрица размера $n \times q$, изначально заполненная нулями.

Для алгоритмов Винограда дополнительно определены:

— $mulH$ — массив из n элементов, вычисляемый по формуле (1.3);

— $mulV$ — массив из q элементов, вычисляемый по формуле (1.4).

2.2.1 Стандартный алгоритм умножения матриц

Схема стандартного алгоритма умножения матриц приведена на рисунке 2.1.

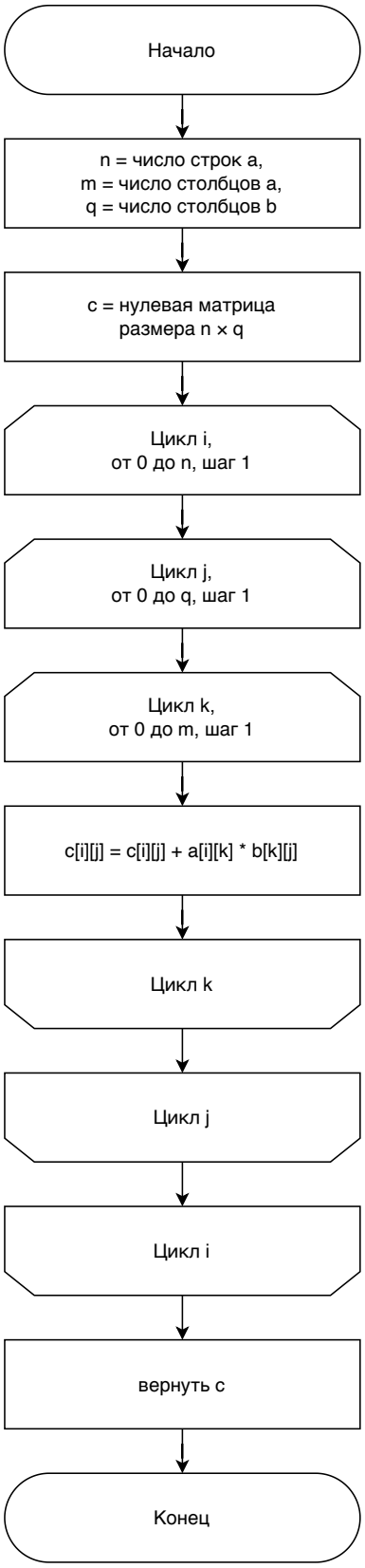


Рисунок 2.1 — Схема стандартного алгоритма умножения матриц

2.2.2 Алгоритм Винограда

Алгоритм состоит из четырёх этапов: вычисление массива $mulH$, вычисление массива $mulV$, вычисление элементов результата по формуле (1.5) и добавление слагаемого $a_{i,m-1} \cdot b_{m-1,j}$ при нечётном m. Схема алгоритма Винограда приведена на рисунках 2.2–2.4.

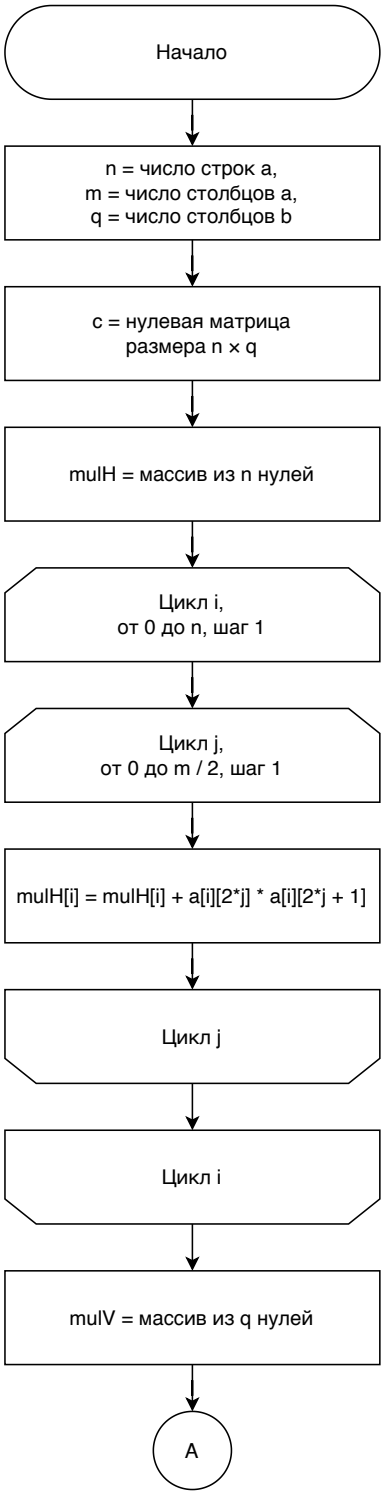


Рисунок 2.2 — Схема алгоритма Винограда, лист 1

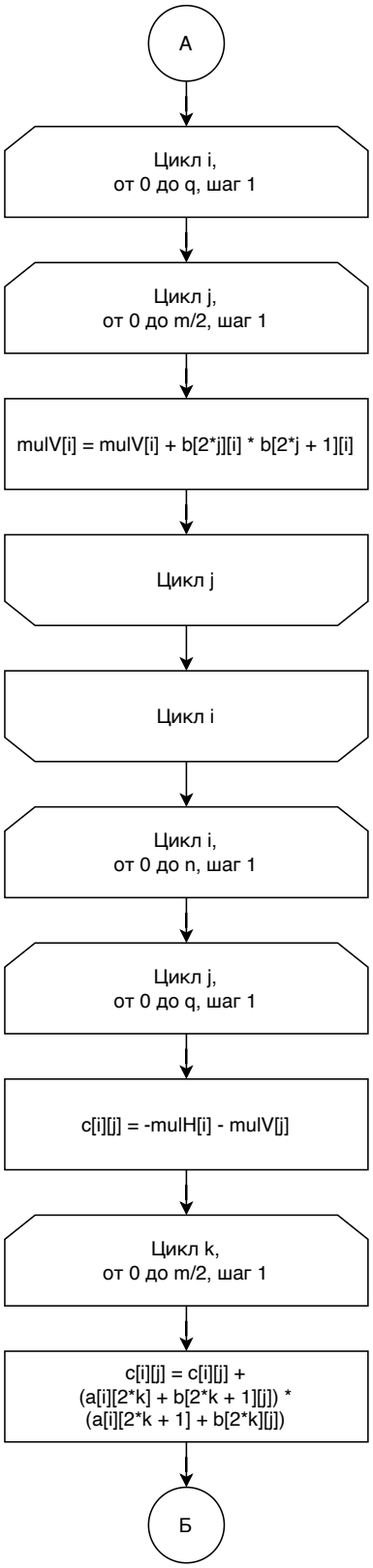


Рисунок 2.3 — Схема алгоритма Винограда, лист 2

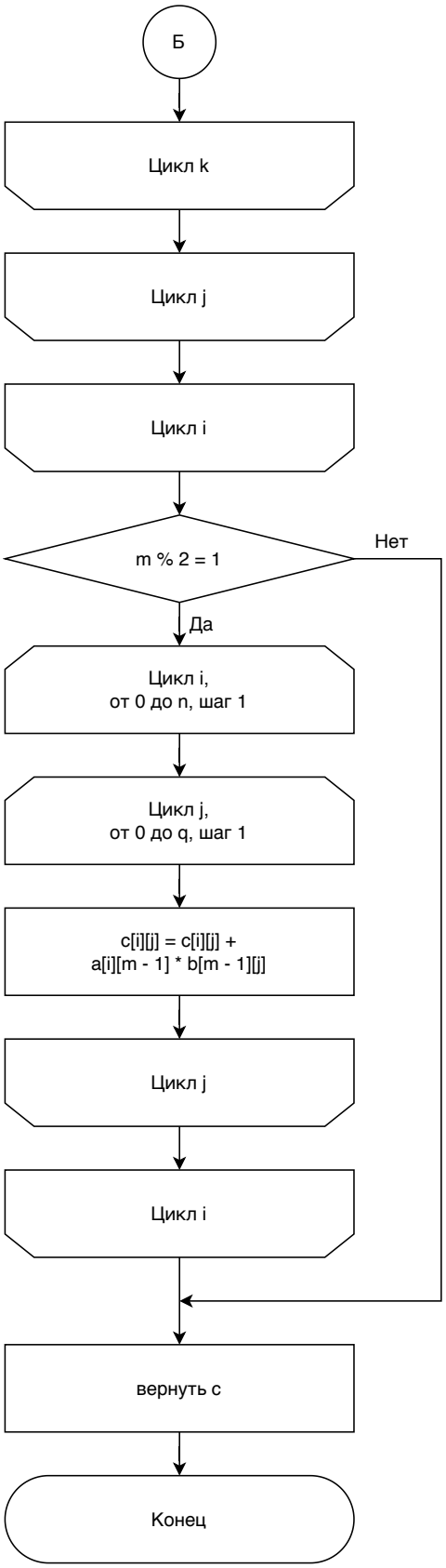


Рисунок 2.4 — Схема алгоритма Винограда, лист 3

2.2.3 Оптимизированный алгоритм Винограда

Согласно варианту к алгоритму Винограда применяются две оптимизации:

— оптимизация А — деление на 2 при вычислении границы циклов по индексу пары заменяется побитовым сдвигом вправо на 1 разряд;

— оптимизация Б — накопление сумм вида $x = x + y$ заменяется составным оператором присваивания $x \mathrel{+}= y$.

Каждая оптимизация применяется к алгоритму Винограда отдельно, что даёт две оптимизированные реализации.

2.2.3.1 Реализация с оптимизацией А

В данной реализации граница циклов по индексу пары $m/2$ вычисляется побитовым сдвигом вправо $m \gg 1$: для неотрицательного целого m оба выражения дают одинаковый результат. Сдвиг применяется во всех трёх циклах по индексу пары: при вычислении $mulH$, $mulV$ и в основном цикле. Схема алгоритма приведена на рисунках 2.5–2.7.

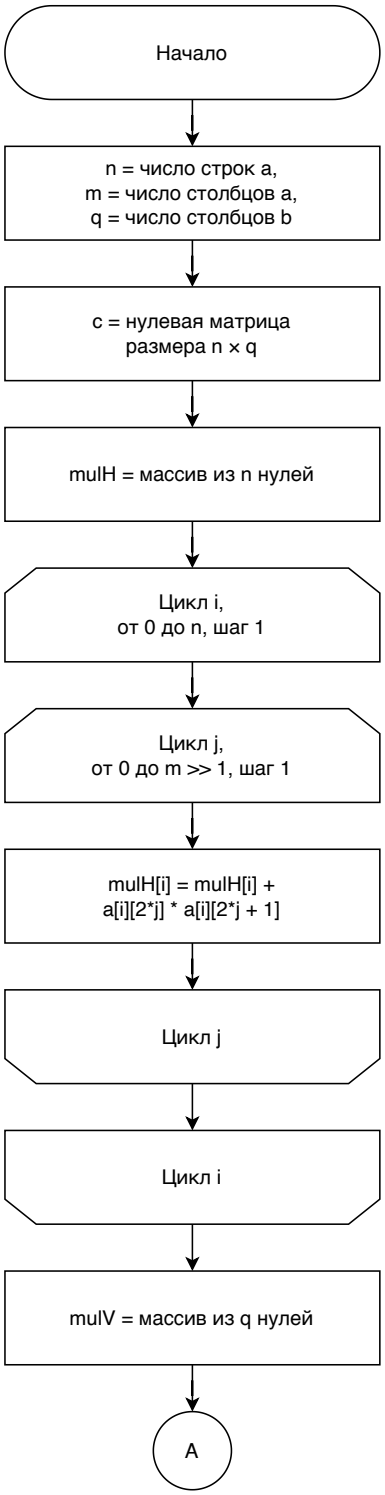


Рисунок 2.5 — Схема алгоритма Винограда с оптимизацией A, лист 1

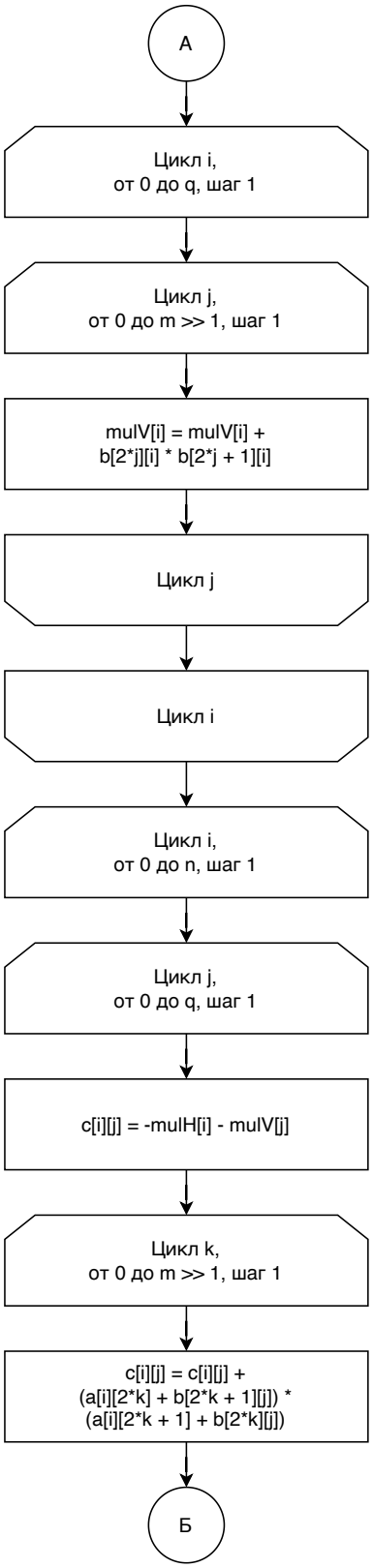


Рисунок 2.6 — Схема алгоритма Винограда с оптимизацией А, лист 2

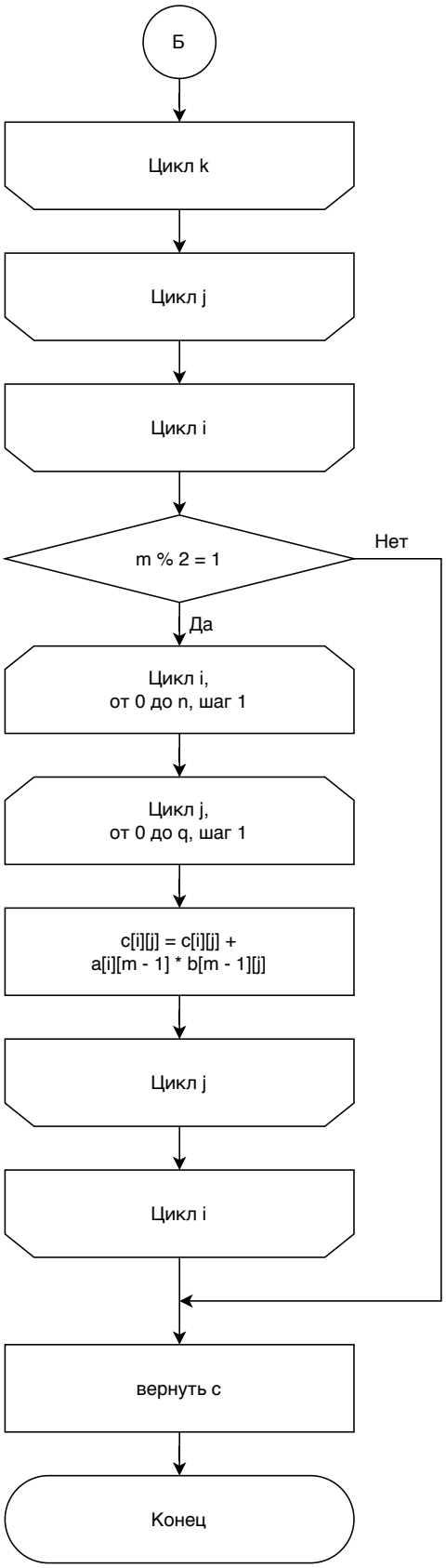


Рисунок 2.7 — Схема алгоритма Винограда с оптимизацией А, лист 3

2.2.3.2 Реализация с оптимизацией Б

В данной реализации все накопления сумм вида $x = x + y$ заменены составным оператором присваивания $x \mathrel{+}= y$: при вычислении $mulH$ и $mulV$, в основном цикле и при добавлении слагаемого для нечётного m. Это избавляет от повторного обращения к накапливаемому элементу массива в правой части присваивания. Схема алгоритма приведена на рисунках 2.8–2.10.

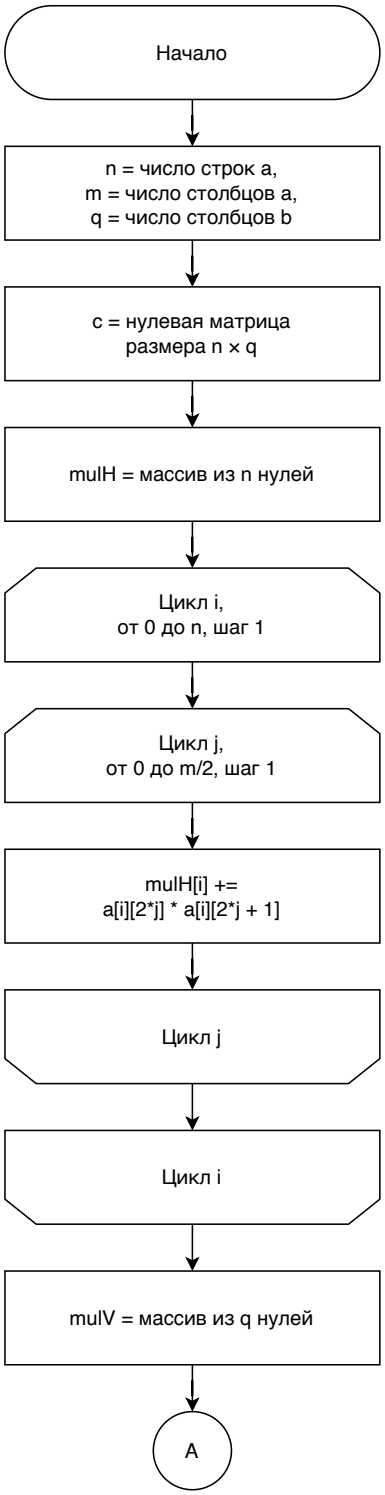


Рисунок 2.8 — Схема алгоритма Винограда с оптимизацией Б, лист 1

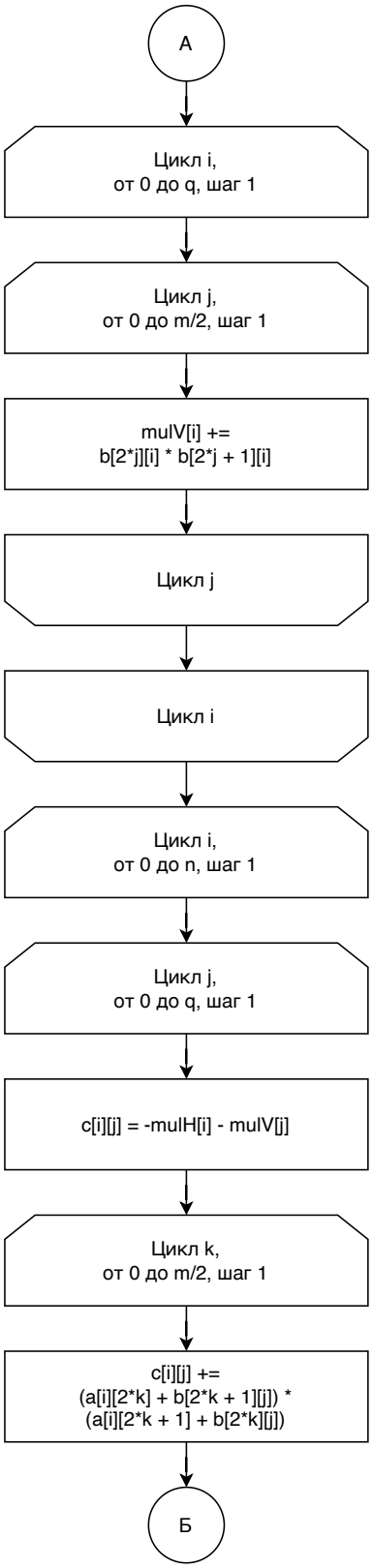


Рисунок 2.9 — Схема алгоритма Винограда с оптимизацией Б, лист 2

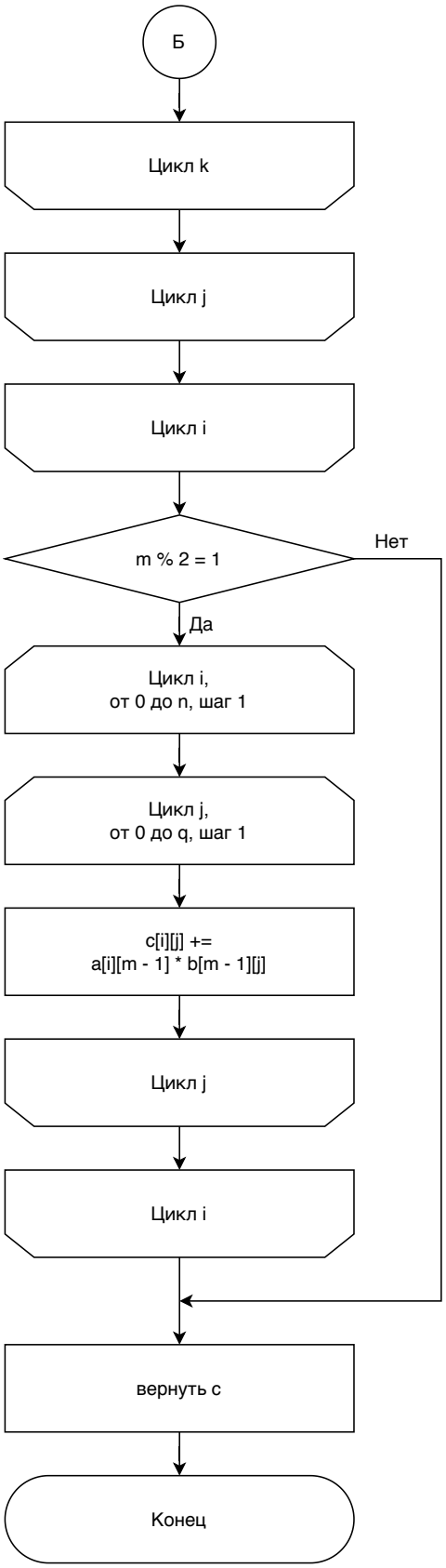


Рисунок 2.10 — Схема алгоритма Винограда с оптимизацией Б, лист 3

3 Технологическая часть

3.1 Средства реализации

В качестве языка программирования был выбран Swift версии 6.4, так как он является языком со статической типизацией и основным языком разработки для операционной системы macOS, на которой выполнялась лабораторная работа. Сборка выполнялась с помощью Swift Package Manager.

При реализации использовались следующие средства:

— тип `Array` стандартной библиотеки Swift — матрица представлена как массив строк `[[Int]]`, вспомогательные массивы $mulH$ и $mulV$ — как `[Int]`;

— функция `clock_gettime_nsec_np` системной библиотеки macOS [3] — для замера процессорного времени (подробнее в разделе 4.4);

— библиотека Swift Testing — для модульного тестирования реализаций.

3.2 Реализация алгоритмов

Тип матрицы и функция создания нулевой матрицы приведены в листинге 3.1, реализации алгоритмов — в листингах 3.2–3.5. Фрагменты кода, которые выводят промежуточные данные только в отладочной сборке (блоки `#if DEBUG`), в листингах опущены.

Листинг 3.1 — Тип матрицы и создание нулевой матрицы

```
package typealias MatrixMultiplyFuncition = (Matrix, Matrix) -> Matrix
package typealias Matrix = [[Int]]

package func matrixInit(_ n: Int, _ m: Int) -> Matrix {
    Array(repeating: Array(repeating: 0, count: m), count: n)
}

package func print(_ matrix: Matrix) {
    for r in matrix {
        print(r)
    }
}
```

Листинг 3.2 — Стандартный алгоритм умножения матриц

```
package func matrixMultiply(a: Matrix, b: Matrix) -> Matrix {
    let n = a.count, m = a[0].count, q = b[0].count
    var c = matrixInit(n, q)

    for i in 0..<n {
        for j in 0..<q {
            for k in 0..<m {
                c[i][j] = c[i][j] + a[i][k] * b[k][j]
            }
        }
    }

    return c
}
```

Листинг 3.3 — Алгоритм Винограда

```
package func winogradMultiply(a: Matrix, b: Matrix) -> Matrix {
    let n = a.count, m = a[0].count, q = b[0].count
    var c = matrixInit(n, q)

    // mulH
    var mulH = Array(repeating: 0, count: n)
    for i in 0..<n {
        for j in 0..<(m/2) {
            mulH[i] = mulH[i] + a[i][2*j] * a[i][2*j + 1]
        }
    }

    // mulV
    var mulV = Array(repeating: 0, count: q)
    for i in 0..<q {
        for j in 0..<(m/2) {
            mulV[i] = mulV[i] + b[2*j][i] * b[2*j + 1][i]
        }
    }
```

Продолжение листинга 3.3

```
    // Основная матрица
    for i in 0..<n {
        for j in 0..<q {
            c[i][j] = -mulH[i] - mulV[j]
            for k in 0..<(m/2) {
                c[i][j] = c[i][j] + (a[i][2*k] + b[2*k + 1][j]) * (a[i][2*k
   + 1] + b[2*k][j])
            }
        }
    }

    // Поправка при M нечетном
    if m % 2 == 1 {
        for i in 0..<n {
            for j in 0..<q {
                c[i][j] = c[i][j] + a[i][m - 1] * b[m - 1][j]
            }
        }
    }

    return c
}
```

Листинг 3.4 — Алгоритм Винограда с оптимизацией A

```
package func winogradMultiplyShift(a: Matrix, b: Matrix) -> Matrix {
    let n = a.count, m = a[0].count, q = b[0].count
    var c = matrixInit(n, q)

    // mulH
    var mulH = Array(repeating: 0, count: n)
    for i in 0..<n {
        for j in 0..<(m>>1) {
            mulH[i] = mulH[i] + a[i][2*j] * a[i][2*j + 1]
        }
    }

    // mulV
    var mulV = Array(repeating: 0, count: q)
    for i in 0..<q {
        for j in 0..<(m>>1) {
            mulV[i] = mulV[i] + b[2*j][i] * b[2*j + 1][i]
        }
    }
```

Продолжение листинга 3.4

```
    // Основная матрица
    for i in 0..<n {
        for j in 0..<q {
            c[i][j] = -mulH[i] - mulV[j]
            for k in 0..<(m>>1) {
                c[i][j] = c[i][j] + (a[i][2*k] + b[2*k + 1][j]) * (a[i][2*k
   + 1] + b[2*k][j])
            }
        }
    }

    // Поправка при M нечетном
    if m % 2 == 1 {
        for i in 0..<n {
            for j in 0..<q {
                c[i][j] = c[i][j] + a[i][m - 1] * b[m - 1][j]
            }
        }
    }

    return c
}
```

Листинг 3.5 — Алгоритм Винограда с оптимизацией Б

```
package func winogradMultiplyAddAsigned(a: Matrix, b: Matrix) -> Matrix {
    let n = a.count, m = a[0].count, q = b[0].count
    var c = matrixInit(n, q)

    // mulH
    var mulH = Array(repeating: 0, count: n)
    for i in 0..<n {
        for j in 0..<(m/2) {
            mulH[i] += a[i][2*j] * a[i][2*j + 1]
        }
    }

    // mulV
    var mulV = Array(repeating: 0, count: q)
    for i in 0..<q {
        for j in 0..<(m/2) {
            mulV[i] += b[2*j][i] * b[2*j + 1][i]
        }
    }
```

Продолжение листинга 3.5

```
    // Основная матрица
    for i in 0..<n {
        for j in 0..<q {
            c[i][j] = -mulH[i] - mulV[j]
            for k in 0..<(m/2) {
                c[i][j] += (a[i][2*k] + b[2*k + 1][j]) * (a[i][2*k + 1] + b
   [2*k][j])
            }
        }
    }

    // Поправка при M нечетном
    if m % 2 == 1 {
        for i in 0..<n {
            for j in 0..<q {
                c[i][j] += a[i][m - 1] * b[m - 1][j]
            }
        }
    }

    return c
}
```

3.3 Тестирование

Тестирование выполнялось методом чёрного ящика. Набор функциональных тестов, применявшийся ко всем четырём реализациям, приведён в таблице 3.1. Тесты 1–10 проверяют результат на заранее вычисленных значениях, тест 11 сравнивает результат каждой реализации с результатом стандартного алгоритма на случайных матрицах с элементами из отрезка $[-50; 50]$. Листинг модульных тестов приведён в приложении А.

Таблица 3.1 — Функциональные тесты

№	**Описание теста**	A	B	**Ожидаемый результат**	**Получ.**
1	Матрицы 1×1	$\begin{pmatrix}3\end{pmatrix}$	$\begin{pmatrix}-4\end{pmatrix}$	$\begin{pmatrix}-12\end{pmatrix}$	Совп.
2	Умножение на единичную матрицу E слева и справа	$\begin{pmatrix}1 & 2 & 3\\4 & 5 & 6\\7 & 8 & 9\end{pmatrix}$	$E_{3\times3}$	$AE = EA = A$	Совп.
3	Умножение на нулевую матрицу	$\begin{pmatrix}1 & 2\\3 & 4\end{pmatrix}$	$\begin{pmatrix}0 & 0\\0 & 0\end{pmatrix}$	$\begin{pmatrix}0 & 0\\0 & 0\end{pmatrix}$	Совп.
4	Квадратные 2×2, чётное m	$\begin{pmatrix}1 & 2\\3 & 4\end{pmatrix}$	$\begin{pmatrix}5 & 6\\7 & 8\end{pmatrix}$	$\begin{pmatrix}19 & 22\\43 & 50\end{pmatrix}$	Совп.
5	Квадратные 3×3, нечётное m	$\begin{pmatrix}1 & 2 & 3\\4 & 5 & 6\\7 & 8 & 9\end{pmatrix}$	$\begin{pmatrix}9 & 8 & 7\\6 & 5 & 4\\3 & 2 & 1\end{pmatrix}$	$\begin{pmatrix}30 & 24 & 18\\84 & 69 & 54\\138 & 114 & 90\end{pmatrix}$	Совп.
6	Прямоугольные 2×3 и 3×2, нечётное m	$\begin{pmatrix}1 & 2 & 3\\4 & 5 & 6\end{pmatrix}$	$\begin{pmatrix}7 & 8\\9 & 10\\11 & 12\end{pmatrix}$	$\begin{pmatrix}58 & 64\\139 & 154\end{pmatrix}$	Совп.
7	Прямоугольные 3×2 и 2×4, чётное m	$\begin{pmatrix}1 & 2\\3 & 4\\5 & 6\end{pmatrix}$	$\begin{pmatrix}1 & 2 & 3 & 4\\5 & 6 & 7 & 8\end{pmatrix}$	$\begin{pmatrix}11 & 14 & 17 & 20\\23 & 30 & 37 & 44\\35 & 46 & 57 & 68\end{pmatrix}$	Совп.
8	Строка на столбец	$\begin{pmatrix}1 & 2 & 3 & 4\end{pmatrix}$	$\begin{pmatrix}1 & 2 & 3 & 4\end{pmatrix}^T$	$\begin{pmatrix}30\end{pmatrix}$	Совп.
9	Столбец на строку, $m = 1$	$\begin{pmatrix}1 & 2 & 3\end{pmatrix}^T$	$\begin{pmatrix}4 & 5\end{pmatrix}$	$\begin{pmatrix}4 & 5\\8 & 10\\12 & 15\end{pmatrix}$	Совп.
10	Отрицательные элементы	$\begin{pmatrix}-1 & 2\\3 & -4\end{pmatrix}$	$\begin{pmatrix}-5 & -6\\7 & 8\end{pmatrix}$	$\begin{pmatrix}19 & 22\\-43 & -50\end{pmatrix}$	Совп.
11	Случайные матрицы, $m = \overline{1,6}$, $n, q \in [1;5]$	Случайная $n \times m$	Случайная $m \times q$	Результат стандартного алгоритма	Совп.

Все тесты пройдены успешно.

4 Исследовательская часть

4.1 Модель вычислений

Для оценки трудоёмкости используется следующая модель вычислений.

1) Операции `=`, `+`, `-` (в том числе унарный минус), `+=`, `-=`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `[]`, `++`, `--`, `<<`, `>>`, `&&`, `||` имеют трудоёмкость 1.
2) Операции `*`, `/`, `%`, `*=`, `/=`, `%=` имеют трудоёмкость 2.
3) Трудоёмкость условного оператора

$$f_{if} = f_{\text{усл}} + \begin{cases} \min(f_A, f_B) & \text{в лучшем случае,} \\ \max(f_A, f_B) & \text{в худшем случае,} \end{cases} \tag{4.1}$$

где $f_{\text{усл}}$ — трудоёмкость вычисления условия, f_A и f_B — трудоёмкости ветвей.

4) Трудоёмкость цикла

$$f_{for} = f_{\text{иниц}} + f_{\text{сравн}} + K \cdot (f_{\text{тела}} + f_{\text{инкр}} + f_{\text{сравн}}), \tag{4.2}$$

где K — число итераций. Для цикла `for i in 0..<x` языка Swift диапазон `0..<x` создаётся один раз перед началом цикла, поэтому вычисление границы x относится к инициализации: $f_{\text{иниц}} = 1 + f_{\text{гр}}$, где $f_{\text{гр}}$ — трудоёмкость вычисления границы. Сравнение и инкремент имеют трудоёмкость 1. Тогда

$$f_{for} = 2 + f_{\text{гр}} + K \cdot (2 + f_{\text{тела}}). \tag{4.3}$$

5) Трудоёмкость вызова функции равна 0.

Выделение памяти под результирующую матрицу и вспомогательные массивы, а также определение размеров матриц в оценке не учитываются, так как выполняются одинаково во всех алгоритмах. Далее обозначено $H = \lfloor m/2 \rfloor$.

4.2 Трудоёмкость алгоритмов

4.2.1 Стандартный алгоритм умножения матриц

Тело внутреннего цикла `c[i][j] = c[i][j] + a[i][k] * b[k][j]` содержит 8 операций индексации, одно умножение, одно сложение и одно присваивание, его трудоёмкость равна $8 + 2 + 1 + 1 = 12$. Границы циклов n, q, m не требуют вычислений. По формуле (4.3)

$$f_{\text{станд}} = 2 + n\big(4 + q(4 + 14m)\big) = 14nmq + 4nq + 4n + 2. \tag{4.4}$$

4.2.2 Алгоритм Винограда

Трудоёмкость алгоритма складывается из трудоёмкостей четырёх этапов:

$$f_{\text{Вин}} = f_{mulH} + f_{mulV} + f_{\text{осн}} + f_{\text{неч}}. \tag{4.5}$$

Граница циклов по индексу пары $m/2$ имеет трудоёмкость $f_{\text{гр}} = 2$.

Тело цикла вычисления $mulH$ `mulH[i] = mulH[i] + a[i][2*j] * a[i][2*j + 1]` содержит 6 операций индексации, 3 умножения, 2 сложения и присваивание: $6 + 6 + 2 + 1 = 15$. Тогда

$$f_{mulH} = 2 + n(2 + 4 + 17H) = 17nH + 6n + 2. \tag{4.6}$$

Цикл вычисления $mulV$ устроен аналогично:

$$f_{mulV} = 17qH + 6q + 2. \tag{4.7}$$

В основном цикле присваивание `c[i][j] = -mulH[i] - mulV[j]` имеет трудоёмкость $2 + 1 + 1 + 1 + 1 + 1 = 7$ (4 индексации, унарный минус, вычитание, присваивание). Тело внутреннего цикла `c[i][j] = c[i][j] + (a[i][2*k] + b[2*k + 1][j]) * (a[i][2*k + 1] + b[2*k][j])` содержит 12 операций индексации, 5 умножений, 5 сложений и присваивание: $12 + 10 + 5 + 1 = 28$. Тогда

$$f_{\text{осн}} = 2 + n\Big(4 + q\big(7 + 4 + 2 + 30H\big)\Big) = 30nqH + 13nq + 4n + 2. \tag{4.8}$$

Проверка условия `m % 2 == 1` имеет трудоёмкость 3, тело цикла `c[i][j] = c[i][j] + a[i][m - 1] * b[m - 1][j]` — $8+2+2+1+1 = 14$:

$$f_{\text{неч}} = 3 + \begin{cases} 0, & m \text{ чётное (лучший случай)}, \\ 16nq + 4n + 2, & m \text{ нечётное (худший случай)}. \end{cases} \tag{4.9}$$

Итого

$$f_{\text{Вин}} = 30nqH + 17H(n+q) + 13nq + 10n + 6q + 9 + \\ + \begin{cases} 0, & m \text{ чётное}, \\ 16nq + 4n + 2, & m \text{ нечётное}. \end{cases} \tag{4.10}$$

4.2.3 Оптимизированный алгоритм Винограда

Оптимизации А и Б не меняют структуру алгоритма Винограда, поэтому трудоёмкость каждой реализации выражается через трудоёмкость базового алгоритма (4.10) с учётом изменённых операций.

4.2.3.1 Реализация с оптимизацией А

Замена $m/2$ на $m \gg 1$ уменьшает трудоёмкость вычисления границы цикла с 2 до 1. Так как граница вычисляется один раз при входе в цикл, выигрыш равен числу входов в циклы по индексу пары: n раз при вычислении $mulH$, q раз при вычислении $mulV$ и nq раз в основном цикле:

$$f_{\text{А}} = f_{\text{Вин}} - (nq + n + q). \tag{4.11}$$

Если бы граница проверялась в условии цикла на каждой итерации, как в цикле `for` языка C, выигрыш составил бы 1 на каждую итерацию, то есть $nqH + nH + qH$. В языке Swift такой эффект отсутствует.

4.2.3.2 Реализация с оптимизацией Б

Составной оператор `+=` избавляет от повторной индексации накапливаемого элемента и от отдельной операции сложения:

— `mulH[i] += a[i][2*j] * a[i][2*j + 1]`: $1 + 1 + 4 + 5 + 2 = 13$ вместо 15;

— `c[i][j] += (a[i][2*k] + b[2*k + 1][j]) * (a[i][2*k + 1] + b[2*k][j])`: 25 вместо 28;

— `c[i][j] += a[i][m - 1] * b[m - 1][j]`: 11 вместо 14.

Тогда

$$f_{\text{Б}} = 27nqH + 15H(n+q) + 13nq + 10n + 6q + 9 + \\ + \begin{cases} 0, & m \text{ чётное}, \\ 13nq + 4n + 2, & m \text{ нечётное}. \end{cases} \tag{4.12}$$

4.2.4 Сводная оценка для квадратных матриц

Для квадратных матриц $n = m = q = N$ подстановка $H = N/2$ при чётном N и $H = (N-1)/2$ при нечётном N в формулы (4.4)–(4.12) даёт результаты, приведённые в таблице 4.1. Полученные формулы были дополнительно проверены непосредственным подсчётом трудоёмкости по модели для $N = \overline{1,29}$.

Таблица 4.1 — Трудоёмкость алгоритмов для квадратных матриц размера $N \times N$

Алгоритм	**Чётное** N	**Нечётное** N
Стандартный	$14N^3 + 4N^2 + 4N + 2$	
Виноград	$15N^3 + 30N^2 + 16N + 9$	$15N^3 + 31N^2 + 3N + 11$
Виноград, опт. А	$15N^3 + 29N^2 + 14N + 9$	$15N^3 + 30N^2 + N + 11$
Виноград, опт. Б	$13{,}5N^3 + 28N^2 + 16N + 9$	$13{,}5N^3 + 27{,}5N^2 + 5N + 11$

Согласно модели, алгоритм Винограда без оптимизаций более трудоёмкий относительно стандартного алгоритма: вдвое меньшее число умножений элементов матриц не компенсирует вычисление индексов $2k$ и $2k+1$ (4 умножения на итерацию) и большее число операций индексации. Оптимизация А влияет только на слагаемые порядка N^2 и практически не меняет трудоёмкость. Оптимизация Б уменьшает коэффициент при старшем члене с 15 до 13,5, и только эта версия алгоритма Винограда по модели оказывается менее трудоёмкой, чем стандартный алгоритм. Все четыре алгоритма имеют кубическую трудоёмкость.

4.3 Характеристики устройства

Исследования проводились на машине со следующими характеристиками:

— ноутбук MacBook Air;

— процессор Apple M5, 10 ядер (4 производительных и 6 энергоэффективных);

— оперативная память: 16 ГБ;

— операционная система: macOS 27.0.1.

4.4 Инструменты замера процессорного времени

Для замера процессорного времени использовалась функция `clock_gettime_nsec_np` системной библиотеки macOS с идентификатором часов `CLOCK_THREAD_CPUTIME_ID` [3]. Функция возвращает значение часов в наносекундах в виде 64-битного беззнакового целого, а часы `CLOCK_THREAD_CPUTIME_ID` учитывают процессорное время (в режиме пользователя и в режиме ядра), затраченное вызывающим потоком. Таким образом, в замер не попадает время, в течение которого поток был вытеснен другими процессами.

Время выполнения одного алгоритма для одного размера определяется как среднее по 10 повторам умножения одних и тех же случайно сгенерированных матриц с элементами из отрезка $[-100; 100]$. Чтобы компилятор не исключил вычисление неиспользуемого результата, результат передаётся в функцию `blackHole`, запрещённую к встраиванию. Замеры выполнялись без оптимизаций компилятора: программа собиралась командой `swift run -c release -Xswiftc -Onone`, где `-Onone` — уровень оптимизации компилятора Swift, аналогичный флагу `-O0` компиляторов C и C++. Конфигурация `release` использована для того, чтобы исключить отладочный вывод алгоритмов (блоки `#if DEBUG`), а флаг `-Onone` — чтобы компилятор не устранял различия между реализациями, которые оцениваются моделью вычислений: повторную индексацию, вычисление индексов $2k$ и $2k+1$, запись накопления суммы. Функция замера `measure` и функция `runTimeMeasurements`, выполняющая массированные замеры, приведены в листинге А.2 приложения А.

4.5 Результаты замеров

Замеры выполнялись для квадратных матриц чётных размеров $N = 10, 20, \ldots, 200$ и нечётных размеров $N = 11, 21, \ldots, 201$. Результаты приведены в таблице 4.2 и на рисунках 4.1 и 4.2.

Таблица 4.2 — Процессорное время умножения квадратных матриц, мс

Чётные размеры					**Нечётные размеры**				
N	Станд.	Виногр.	Опт. А	Опт. Б	N	Станд.	Виногр.	Опт. А	Опт. Б
10	0,49	0,33	0,28	0,25	11	0,39	0,26	0,26	0,26
20	1,85	1,01	0,88	0,76	21	1,14	0,72	0,68	0,63
30	2,87	1,88	1,87	1,66	31	3,14	2,09	2,06	1,93
40	6,75	4,33	4,23	3,96	41	7,13	4,64	4,66	4,33
50	12,91	8,17	8,15	7,49	51	13,71	9,00	8,83	8,13
60	22,33	14,13	14,08	12,94	61	23,32	14,99	14,78	13,74
70	35,42	22,23	22,02	20,35	71	36,90	23,42	23,33	21,50
80	52,85	32,73	32,45	30,59	81	54,73	34,35	34,29	31,37
90	74,83	46,29	46,48	42,68	91	77,56	48,54	48,44	44,39
100	104,22	64,68	63,04	59,32	101	109,60	66,97	66,78	60,61
110	136,70	84,73	84,30	77,10	111	139,87	86,41	86,96	80,08
120	177,60	109,35	109,27	101,30	121	185,88	113,15	112,57	103,01
130	225,46	142,34	140,95	127,73	131	228,81	140,83	141,41	130,29
140	282,83	170,52	171,76	159,86	141	287,71	177,78	177,46	161,53
150	343,73	214,89	213,48	195,25	151	355,86	219,45	218,27	199,57
160	420,89	258,71	256,88	235,47	161	426,04	262,36	263,02	241,98
170	504,04	311,99	307,68	285,66	171	518,67	315,70	319,42	290,29
180	602,37	370,60	366,86	339,14	181	612,90	379,10	374,29	344,84
190	706,35	435,33	431,90	395,49	191	707,82	435,70	435,78	400,61
200	816,33	499,13	498,46	459,13	201	824,38	507,70	503,39	458,74

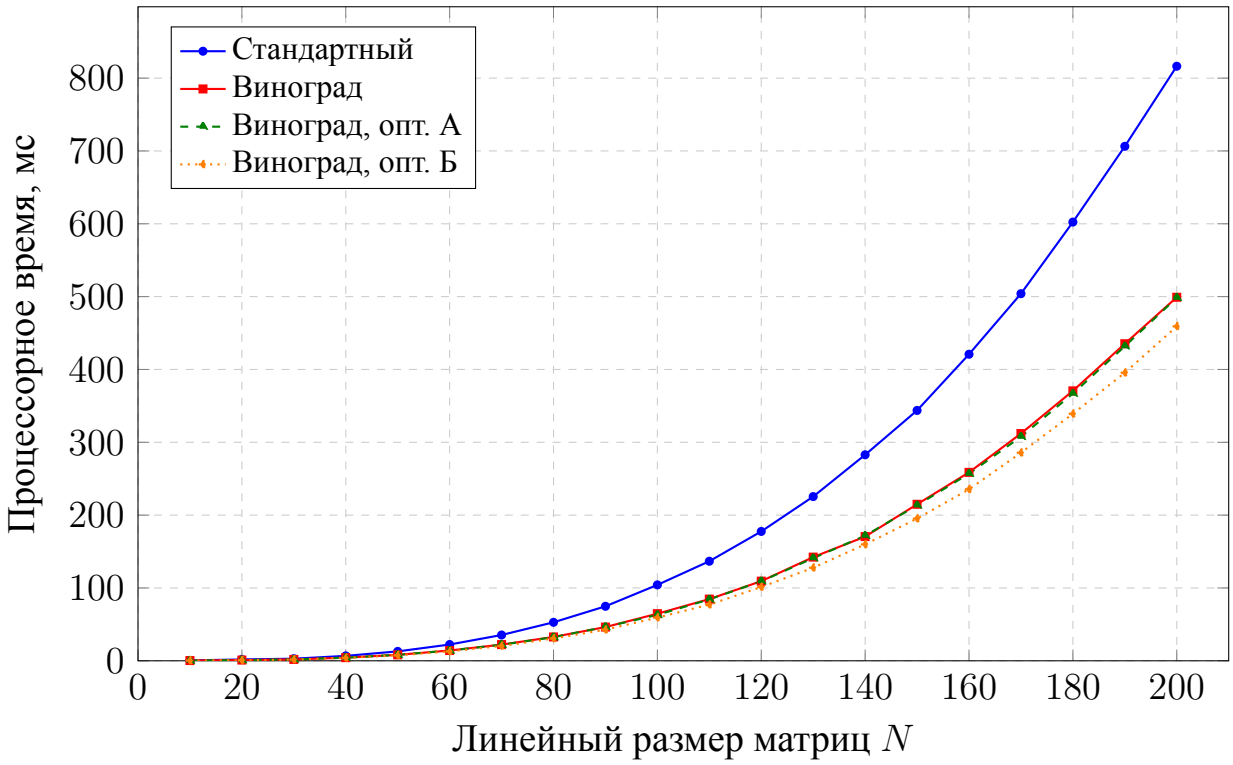


Рисунок 4.1 — Зависимость процессорного времени от линейного размера матриц, чётные размеры

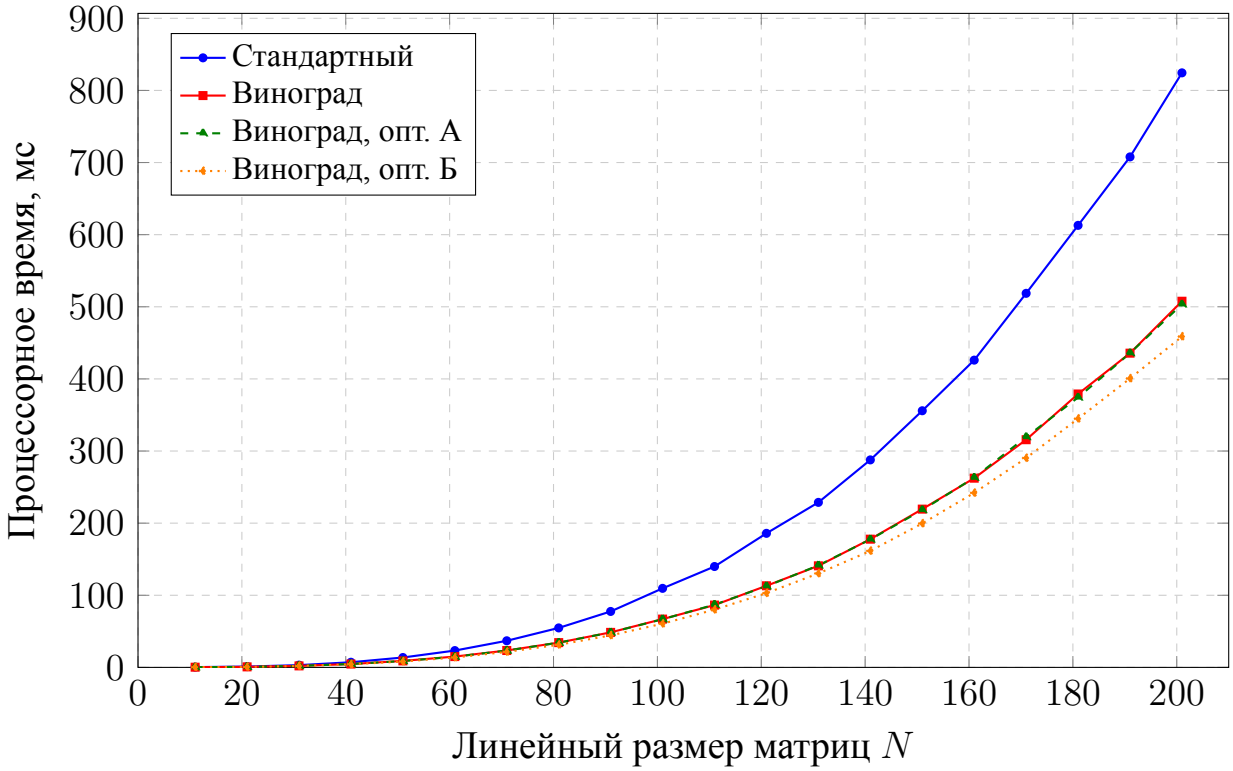


Рисунок 4.2 — Зависимость процессорного времени от линейного размера матриц, нечётные размеры

4.6 Сравнительный анализ

Для сравнения алгоритмов используется относительное значение трудоёмкости и времени работы алгоритма X по отношению к алгоритму Y:

$$f_{\text{отн}} = \frac{f_X(N)}{f_Y(N)}, \qquad t_{\text{отн}} = \frac{t_X(N)}{t_Y(N)}, \tag{4.13}$$

где $f_X(N)$, $f_Y(N)$ — трудоёмкости алгоритмов по таблице 4.1, $t_X(N)$, $t_Y(N)$ — измеренное процессорное время по таблице 4.2. Значение меньше единицы означает, что алгоритм X эффективнее алгоритма Y. В таблице 4.3 приведены значения при $N = 200$ по отношению к стандартному алгоритму.

Таблица 4.3 — Сравнение алгоритмов со стандартным алгоритмом при $N = 200$

Алгоритм	**Трудоёмкость**	$f_{\text{отн}}$	**Время, мс**	$t_{\text{отн}}$
Стандартный	112 160 802	1,000	816,33	1,000
Виноград	121 203 209	1,081	499,13	0,611
Виноград, опт. А	121 162 809	1,080	498,46	0,611
Виноград, опт. Б	109 123 209	0,973	459,13	0,562

При увеличении N вдвое, со 100 до 200, время работы каждого алгоритма возрастает в 7,7–7,9 раза, то есть примерно в $2^3 = 8$ раз, что соответствует кубической трудоёмкости.

По данным замеров все версии алгоритма Винограда работают быстрее стандартного алгоритма при всех исследованных размерах: при $N = 200$ базовая версия быстрее в $1/0{,}611 \approx 1{,}6$ раза, версия с оптимизацией Б — в $1/0{,}562 \approx 1{,}8$ раза. Модель вычислений, напротив, предсказывает, что алгоритм Винограда на 8 % трудоёмче стандартного ($f_{\text{отн}} = 1{,}081$).

Расхождение связано с тем, что в модели обращение к элементу массива `[]` имеет такую же трудоёмкость, как сложение. В реализации матрица хранится как массив массивов `[[Int]]`, поэтому обращение к элементу $c_{i,j}$ требует обращения сначала к строке c_i, а затем к элементу строки. При изменении элемента массива Swift дополнительно проверяет, не используется ли массив другими переменными, и при необходимости копирует его (механизм копирования при записи, copy-on-write). Поэтому чтение и особенно изменение элемента матрицы обходятся дороже сложения или умножения целых чи-

сел. Число обращений к элементам матриц во внутреннем цикле приведено в таблице 4.4.

Таблица 4.4 — Число обращений к элементам матриц во внутреннем цикле

Алгоритм	**Чтений** a**,** b	**Чтений** c	**Изменений** c
Стандартный	$2N^3$	N^3	N^3
Виноград, Виноград с опт. А	$2N^3$	$N^3/2$	$N^3/2$
Виноград с опт. Б	$2N^3$	—	$N^3/2$

Алгоритм Винограда обрабатывает за одну итерацию внутреннего цикла два слагаемых суммы, поэтому читает и изменяет элемент $c_{i,j}$ вдвое реже, чем стандартный алгоритм. Именно за счёт этого он работает быстрее, несмотря на дополнительные операции вычисления индексов $2k$ и $2k + 1$.

Оптимизации сравниваются с базовой версией алгоритма Винограда по формуле (4.13), где Y — алгоритм Винограда без оптимизаций. При $N = 200$:

— для оптимизации А $f_{\text{отн}} = 121\,162\,809/121\,203\,209 \approx 1{,}000$ и $t_{\text{отн}} = 498{,}46/499{,}13 \approx 0{,}999$ — оптимизация практически не влияет на трудоёмкость и время работы, так как граница цикла `0..<(m / 2)` вычисляется один раз при входе в цикл, а не на каждой итерации;

— для оптимизации Б $f_{\text{отн}} = 109\,123\,209/121\,203\,209 \approx 0{,}900$ и $t_{\text{отн}} = 459{,}13/499{,}13 \approx 0{,}920$ — оптимизация уменьшает время работы на 8 %, что близко к предсказанным моделью 10 %: оператор `+=` изменяет элемент $c_{i,j}$ без отдельного чтения в правой части присваивания.

Для нечётного размера $N = 201$ значения $t_{\text{отн}}$ по отношению к стандартному алгоритму составляют $0{,}616$ для алгоритма Винограда, $0{,}611$ для версии с оптимизацией А и $0{,}556$ для версии с оптимизацией Б, то есть совпадают со значениями для чётного размера. Дополнительный этап алгоритма Винограда для нечётного m имеет трудоёмкость порядка N^2 и на соотношение алгоритмов не влияет.

4.7 Вывод

По результатам сравнительного анализа можно сделать следующие выводы:

— все рассмотренные алгоритмы имеют кубическую трудоёмкость: при увеличении N вдвое время работы возрастает примерно в 8 раз;

— все версии алгоритма Винограда работают быстрее стандартного алгоритма: при $N = 200$ базовая версия — в 1,6 раза, версия с оптимизацией Б — в 1,8 раза, так как алгоритм Винограда вдвое реже обращается к накапливаемому элементу результирующей матрицы;

— модель вычислений, в которой обращение к элементу массива имеет такую же трудоёмкость, как сложение, не предсказывает преимущество алгоритма Винограда над стандартным алгоритмом ($15N^3$ против $14N^3$ в старшем члене);

— оптимизация А не даёт выигрыша, что согласуется с моделью: граница цикла вычисляется один раз при входе в цикл;

— оптимизация Б уменьшает время работы алгоритма Винограда на 8 %, что близко к предсказанным моделью 10 %;

— различий в соотношении алгоритмов для чётных и нечётных размеров матриц не обнаружено;

— из рассмотренных реализаций для умножения матриц предпочтителен алгоритм Винограда с оптимизацией Б; алгоритм Винограда требует дополнительной памяти под массивы $mulH$ и $mulV$ размера $n + q$.

ЗАКЛЮЧЕНИЕ

Цель работы достигнута: выполнен сравнительный анализ алгоритмов умножения матриц — стандартного алгоритма, алгоритма Винограда и двух его оптимизированных версий — по трудоёмкости и по данным замеров процессорного времени.

Все задачи решены:

1) описана решаемая согласно варианту задача умножения матриц;
2) разработаны стандартный алгоритм умножения матриц, алгоритм Винограда и две оптимизированные по варианту версии алгоритма Винограда: с заменой деления на 2 побитовым сдвигом (оптимизация А) и с использованием составного оператора += (оптимизация Б);
3) разработанные алгоритмы описаны схемами алгоритмов;
4) описаны требования к программному обеспечению;
5) описаны средства реализации алгоритмов, в качестве языка программирования выбран Swift;
6) разработанные алгоритмы реализованы;
7) выполнено тестирование реализаций алгоритмов, все реализации прошли функциональные тесты;
8) описана модель вычислений и выполнен теоретический расчёт трудоёмкости всех алгоритмов;
9) описаны инструменты замера процессорного времени: функция `clock_gettime_nsec_np` с часами `CLOCK_THREAD_CPUTIME_ID`;
10) выполнены массированные замеры процессорного времени для квадратных матриц чётных размеров от 10 до 200 и нечётных размеров от 11 до 201;
11) выполнен сравнительный анализ алгоритмов по трудоёмкости и по данным замеров.

По результатам сравнительного анализа можно сделать следующие выводы:

— все рассмотренные алгоритмы имеют кубическую трудоёмкость, что подтверждается замерами;

— по данным замеров для сборки без оптимизаций компилятора все версии алгоритма Винограда работают быстрее стандартного алгорит-

ма: базовая версия — в $1{,}6$ раза, версия с оптимизацией Б — в $1{,}8$ раза при $N = 200$;

— преимущество алгоритма Винограда объясняется вдвое меньшим числом обращений к накапливаемому элементу результирующей матрицы; модель вычислений, в которой обращение к элементу массива имеет такую же трудоёмкость, как сложение, это преимущество не предсказывает ($15N^3$ против $14N^3$ в старшем члене);

— оптимизация А (замена деления на 2 сдвигом) не даёт измеримого выигрыша, так как граница цикла в Swift вычисляется один раз при входе в цикл;

— оптимизация Б (составной оператор +=) уменьшает время работы алгоритма Винограда на 8 %, что близко к предсказанным моделью 10 %;

— различий в соотношении алгоритмов для чётных и нечётных размеров матриц не обнаружено.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Laaksonen A. Guide to Competitive Programming: Learning and Improving Algorithms Through Contests. — 3rd ed. — Cham : Springer, 2024. — 349 p. — ISBN: 978-3-031-61793-5.

2. Winograd S. A New Algorithm for Inner Product // IEEE Transactions on Computers. — 1968. — Vol. C-17, no. 7. — P. 693–694.

3. clock_gettime(3) — macOS Library Functions Manual. — Apple Open Source. — Режим доступа: `https://opensource.apple.com/source/Libc/Libc-1244.1.7/gen/clock_gettime.3.auto.html`.

ПРИЛОЖЕНИЕ А

Листинги программы

Листинг А.1 — Модульные тесты реализаций алгоритмов

```
import Testing
@testable import MatrixMultiply

struct NamedAlgorithm: CustomTestStringConvertible, Sendable {
    let name: String
    let algo: @Sendable (Matrix, Matrix) -> Matrix

    var testDescription: String { name }
}

let algorithms: [NamedAlgorithm] = [
    NamedAlgorithm(name: "Классический", algo: matrixMultiply),
    NamedAlgorithm(name: "Виноград", algo: winogradMultiply),
    NamedAlgorithm(name: "Виноград >>1", algo: winogradMultiplyShift),
    NamedAlgorithm(name: "Виноград +=", algo: winogradMultiplyAddAsigned),
]

@Suite("Умножение матриц")
struct MatrixMultiplyTests {

    @Test("Матрицы 1x1", arguments: algorithms)
    func oneByOne(_ a: NamedAlgorithm) {
        #expect(a.algo([[3]], [[-4]]) == [[-12]])
    }
```

Продолжение листинга А.1

```
@Test("Умножение на единичную матрицу", arguments: algorithms)
func identity(_ a: NamedAlgorithm) {
    let m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
    let e = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]
    #expect(a.algo(m, e) == m)
    #expect(a.algo(e, m) == m)
}

@Test("Умножение на нулевую матрицу", arguments: algorithms)
func zero(_ a: NamedAlgorithm) {
    let m = [[1, 2], [3, 4]]
    let z = [[0, 0], [0, 0]]
    #expect(a.algo(m, z) == z)
}

@Test("Квадратные 2x2 (чётное M)", arguments: algorithms)
func square2x2(_ a: NamedAlgorithm) {
    #expect(a.algo([[1, 2], [3, 4]], [[5, 6], [7, 8]]) == [[19, 22],
[43, 50]])
}

@Test("Квадратные 3x3 (нечётное M)", arguments: algorithms)
func square3x3(_ a: NamedAlgorithm) {
    let m1 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
    let m2 = [[9, 8, 7], [6, 5, 4], [3, 2, 1]]
    #expect(a.algo(m1, m2) == [[30, 24, 18], [84, 69, 54], [138, 114,
90]])
}
```

Продолжение листинга А.1

```
 @Test("Прямоугольные 2x3 * 3x2 (нечётное M)", arguments: algorithms)
 func rectangularOdd(_ a: NamedAlgorithm) {
     #expect(a.algo([[1, 2, 3], [4, 5, 6]], [[7, 8], [9, 10], [11, 12]])
== [[58, 64], [139, 154]])
 }

 @Test("Прямоугольные 3x2 * 2x4 (чётное M)", arguments: algorithms)
 func rectangularEven(_ a: NamedAlgorithm) {
     let m1 = [[1, 2], [3, 4], [5, 6]]
     let m2 = [[1, 2, 3, 4], [5, 6, 7, 8]]
     #expect(a.algo(m1, m2) == [[11, 14, 17, 20], [23, 30, 37, 44], [35,
46, 57, 68]])
 }

 @Test("Строка на столбец", arguments: algorithms)
 func rowByColumn(_ a: NamedAlgorithm) {
     #expect(a.algo([[1, 2, 3, 4]], [[1], [2], [3], [4]]) == [[30]])
 }

 @Test("Столбец на строку", arguments: algorithms)
 func columnByRow(_ a: NamedAlgorithm) {
     #expect(a.algo([[1], [2], [3]], [[4, 5]]) == [[4, 5], [8, 10], [12,
15]])
 }

 @Test("Отрицательные элементы", arguments: algorithms)
 func negative(_ a: NamedAlgorithm) {
     #expect(a.algo([[-1, 2], [3, -4]], [[-5, -6], [7, 8]]) == [[19, 22],
  [-43, -50]])
 }
```

Продолжение листинга А.1

```
    @Test("Совпадение с классическим на случайных
    матрицах", arguments: algorithms, 1...6)
    func randomAgainstClassic(_ a: NamedAlgorithm, _ m: Int) {
        let n = Int.random(in: 1...5), q = Int.random(in: 1...5)
        let m1 = (0..<n).map { _ in (0..<m).map { _ in Int.random(in:
    -50...50) } }
        let m2 = (0..<m).map { _ in (0..<q).map { _ in Int.random(in:
    -50...50) } }
        #expect(a.algo(m1, m2) == matrixMultiply(a: m1, b: m2))
    }
}
```

Листинг А.2 — Замеры процессорного времени

```
import MatrixMultiply

/// Размеры квадратных матриц: чётные 10, 20, ..., 200 и
    нечётные 11, 21, ..., 201
let measureSizes = stride(from: 10, through: 200, by: 10).flatMap { [$0, $0
    + 1] }
/// Число повторов умножения для одного размера, итоговое время — среднее
let measureRepeats = 10

/// Каталог с результатами: lab-02/TimeTestResult (рядом с папкой code)
let resultDirectory = URL(fileURLWithPath: #filePath)
    .deletingLastPathComponent() // lab02
    .deletingLastPathComponent() // Sources
    .deletingLastPathComponent() // code
    .deletingLastPathComponent() // lab-02
    .appendingPathComponent("TimeTestResult")

/// Процессорное время текущего потока в наносекундах
func cpuTimeNs() -> UInt64 {
    clock_gettime_nsec_np(CLOCK_THREAD_CPUTIME_ID)
}

/// Не даёт компилятору выбросить вычисление результата
@inline(never)
func blackHole(_ matrix: Matrix) {}
```

Продолжение листинга А.2

```
/// Среднее процессорное время одного умножения (нс)
func measure(_ algo: MatrixMultiplyFuncition, _ a: Matrix, _ b: Matrix,
   repeats: Int) -> UInt64 {
    var total: UInt64 = 0
    for _ in 0..<repeats {
        let start = cpuTimeNs()
        let c = algo(a, b)
        let end = cpuTimeNs()
        blackHole(c)
        total += end - start
    }
    return total / UInt64(repeats)
}

/// Выполняет замеры для всех алгоритмов и пишет по CSV-файлу на алгоритм
func runTimeMeasurements(_ algorithms: [Algorithm]) {

    // times[i][j] — время алгоритма i на размере measureSizes[j]
    var times = Array(repeating: [UInt64](), count: algorithms.count)

    print("n".padding(toLength: 6, withPad: " ", startingAt: 0)
          + algorithms.map { $0.file.padding(toLength: 24, withPad: " ",
   startingAt: 0) }.joined())
```

Продолжение листинга А.2

```
    for n in measureSizes {
        let a = randomMatrix(n, n)
        let b = randomMatrix(n, n)

        var line = String(n).padding(toLength: 6, withPad: " ", startingAt:
    0)
        for (i, algorithm) in algorithms.enumerated() {
            let t = measure(algorithm.algo, a, b, repeats: measureRepeats)
            times[i].append(t)
            line += "\(t) нс".padding(toLength: 24, withPad: " ", startingAt
    : 0)
        }
        print(line)
    }

    do {
        try FileManager.default.createDirectory(at: resultDirectory,
    withIntermediateDirectories: true)
        for (i, algorithm) in algorithms.enumerated() {
            var csv = "size,time_ns\n"
            for (n, t) in zip(measureSizes, times[i]) {
                csv += "\(n),\(t)\n"
            }
            let url = resultDirectory.appendingPathComponent("\(algorithm.
    file).csv")
            try csv.write(to: url, atomically: true, encoding: .utf8)
        }
        print("Результаты сохранены в \(resultDirectory.path)")
    } catch {
        print("Ошибка записи результатов: \(error)")
    }
}
```